



# Rescheduling Involving Deliver Times on a Single Machine due to the Arrival of New Jobs with Rejection

Yi-Xin Ren<sup>1</sup> · Shan-Shan Yu<sup>1</sup> · Le-Shan Tan<sup>2</sup> · Wen-Chang Luo<sup>1</sup>

Received: 27 October 2024 / Accepted: 15 April 2025

© Operations Research Society of China, Periodicals Agency of Shanghai University, Science Press, and Springer-Verlag GmbH Germany, part of Springer Nature 2025

## Abstract

This paper addresses a rescheduling problem where a set of original jobs has already been scheduled to optimize a specific objective on a single machine. However, before processing begins, a set of new jobs arrives and should be inserted into the original jobs with minimal disruption to the planned schedule, which creates a necessity for rescheduling. The disruption is quantified by the maximum deviation in completion time for any original job between the planned and the rescheduled sequences. To ensure an acceptable service level for the original jobs, the rescheduling process allows for the rejection of some new jobs, with each rejected job paying a corresponding rejection cost. We explore three general models in which the maximum completion time disruption is considered both as a constraint and as a component of the cost objective. The primary scheduling objective is to minimize the maximum delivery completion time. The overall cost objective is to minimize the sum of the scheduling objective for the original and newly accepted jobs, the maximum disruption penalty, and the total rejection cost. We present a corresponding dynamic programming exact algorithm that runs in pseudo-polynomial time. Additionally, by employing a trimming technique, we propose a fully polynomial-time approximation scheme.

**Keywords** Rescheduling · Single machine · Job rejection · Dynamic programming · Approximation scheme · Delivery time

This work was supported by the National Natural Science Foundation of China (No. 12431702) and Ningbo Natural Science Foundation (No. 2024J195).

✉ Wen-Chang Luo  
luowenchang@163.com

Yi-Xin Ren  
1159847426@qq.com

Shan-Shan Yu  
2360891699@qq.com

Le-Shan Tan  
53553411@qq.com

<sup>1</sup> School of Mathematics and Statistics, Ningbo University, Ningbo 315211, Zhejiang, China

<sup>2</sup> Faculty of Science and Engineering, University of Nottingham, Ningbo 315100, Zhejiang, China

## 1 Introduction

Rescheduling, which involves adjusting a planned schedule to accommodate unforeseen disruptive events, has been extensively studied in industries such as manufacturing and healthcare [1–3, 19, 20]. Common disruptive events include the arrival of new orders [8], delays in some jobs [7], and periods of machine unavailability due to breakdowns [10]. In this work, we focus on rescheduling when unexpected jobs, referred to as new jobs, must be inserted into an already planned schedule consisting of known jobs, referred to as original jobs. To limit disruption to the planned schedule and maintain an acceptable service level, the decision maker may reject some of the new jobs in order to optimize a specified objective. Our problem falls within the scope of rescheduling for new jobs with job rejection.

Various rescheduling problems involving new jobs have been widely investigated. Hall and Potts [8] addressed several single-machine rescheduling problems involving new jobs, focusing on minimizing total completion time or maximum lateness, subject to disruption constraints or costs. They derived polynomial-time algorithms or proved NP-hardness for the problems considered. Teghem and Tuytens [14] further examined these problems by exploring the enumeration of Pareto-optimal solutions. Zhao et al. [21] introduced a more generalized disruption criterion by imposing a maximum allowable disruption for each original job. They proved strong NP-hardness for minimizing the maximum lateness and presented a simple 2-approximation polynomial-time algorithm. For minimizing total completion time, they also provided a strong NP-hardness proof and developed a heuristic algorithm. Additionally, by relaxing the maximum allowable disruptions, they proposed a super-optimal dual-approximation polynomial-time algorithm. Hall et al. [9] considered rescheduling with multiple sets of new jobs arriving at different times. For minimizing maximum lateness subject to a customer service requirement, modeled as a limit on the maximum change in completion time for the original jobs, they developed approximation algorithms and analyzed their worst-case performances. They also proposed a branch-and-bound algorithm and conducted numerical experiments on instances with up to 1 000 jobs. For the related surveys up to 2009, we refer readers to Vieira et al. [17] and Ouelhadj and Petrovic [13]. After that, Guo et al. [5] examined a single-machine rescheduling problem for newly arrived rework jobs, aiming to minimize total waiting time while achieving energy savings. They formulated the problem as a mixed-integer linear program and proved its NP-hardness. They also developed a pseudo-polynomial optimal algorithm for a special case, alongside heuristic and genetic algorithms with adaptive local search for the general case, demonstrating their effectiveness through computational experiments. Zhang [22] focused on rescheduling problems involving the arrival of unexpected new jobs, aiming to minimize maximum weighted tardiness costs while limiting disruptions to the original schedule. They proved the problem's strong NP-hardness and presented a 2-approximation polynomial-time algorithm, as well as three local search strategies to improve the approximate solution, followed by numerical experiments to evaluate their performance. Ren et al. [15] studied a rescheduling problem for minimizing

maximum lateness, with a constraint on total time disruption, and presented the first exact algorithm to solve the problem with an additional no-idle constraint. Wang et al. [18] were among the first to integrate rescheduling for new jobs with job rejection into a unified model. They described a multi-objective dynamic rescheduling problem in response to continuous job arrivals, under the assumption that jobs could be rejected and that processing times could be controlled with additional resource allocation. The integrated cost function considered total completion time, total rejection cost, and resource consumption cost, with disruption penalties measured by total virtual tardiness, where the completion time of a job in the original schedule was treated as its implied due date. They developed a genetic algorithm incorporating a directed search strategy to find approximate Pareto-optimal solutions for the combined scheduling and disruption costs. Fang et al. [12] studied rescheduling with job rejection to minimize a linear combination of total weighted completion time, maximum disruption, and rejection cost. They proved NP-hardness for the unit weight case and developed an exact dynamic programming algorithm, as well as a fully polynomial-time approximation scheme (FPTAS). Their proposed algorithms were demonstrated to be efficient through computational experiments. Lendl et al. [11] studied rescheduling problems due to the arrival of new orders in which these new jobs should be integrated in a proper way in the existing schedule of the so-called old jobs, with the aim of minimizing an objective function for the joint set of jobs under the constraint that each old job is not allowed to deviate from its original completion time by more than a certain threshold. For the objective of minimizing the total weighted completion time, they showed that the resulting rescheduling problem is weakly NP-hard and derived an exact dynamic programming algorithm with pseudo-polynomial running time. A fully polynomial-time approximation scheme was obtained from the dynamic program by three different scaling and reduction steps. For the objective of minimizing the number of late jobs, a strong NP-hardness proof was given. Renier et al. [16] presented a whole classification of rescheduling problems, with several objective functions and two time disruption constraints, based on the presence of idle times in optimal solutions. The need to insert idle times was shown to be caused by a disruption of the original order of old jobs, which generally makes these problems more difficult to solve than problems where the original order of old jobs is preserved. Several examples and properties related to the insertion of idle times were provided, along with complexity results for five problem variants.

In this paper, we study a novel rescheduling problem where a set of original jobs has already been scheduled on a single machine to minimize the maximum delivery completion time. However, before formal processing begins, a set of new jobs arrives and should be inserted into the original schedule without excessively disrupting the planned schedule, with disruption measured as the maximum deviation in completion time for any given original job between the planned and adjusted schedules. To maintain an acceptable service level for the original jobs, the decision maker may reject some new jobs, incurring rejection costs. We examine three general models where the disruption is treated both as a constraint and as part of the cost objective, with the primary scheduling objective being the minimization of the maximum delivery completion time. The overall cost objective is to minimize the sum of the scheduling

objective for the original and accepted new jobs, the disruption penalty, and the total rejection cost.

This paper is organized as follows: In Sect. 2, we formally define our models. In Sect. 3, we prove the NP-hardness of the problem. Section 4 derives the structural properties and presents a dynamic programming exact algorithm for the models. In Sect. 5, we propose a fully polynomial-time approximation scheme. Finally, Sect. 6 concludes the paper and offers suggestions for future research directions.

## 2 Problem Definition and Notations

In this section, we formally define the studied rescheduling problem as follows.

Let  $\mathcal{J}_O = \{1, 2, \dots, n_O\}$  represent a set of original non-preemptive jobs to be processed on a single machine. Before processing begins, these original jobs have been arranged in an optimal planned schedule to minimize a given objective function, with no-idle time between the jobs. A set of new non-preemptive jobs,  $\mathcal{J}_N = \{n_O + 1, n_O + 2, \dots, n_O + n_N\}$ , arrives at a later time and must be inserted into the planned schedule, which creates a necessity for rescheduling. To maintain an acceptable service level for the original jobs, the decision maker has the option to reject some new jobs during the rescheduling process. The overall objective of the rescheduling is to minimize the total operational cost, which comprises the scheduling objective, the disruption penalty, and the rejection cost.

To describe the problem more precisely, we define  $\mathcal{J} = \mathcal{J}_O \cup \mathcal{J}_N$  and  $n = n_O + n_N$ . Let  $p_j$  denote the processing time and  $q_j$  denote the delivery time of job  $j$  for  $j = 1, 2, \dots, n$ , respectively, and let  $e_j$  denote the rejection cost of new job  $j$  for  $j = n_O + 1, n_O + 2, \dots, n_O + n_N$ . As stated in Hall and Potts [8], all input values are assumed to be known, nonnegative integers. The original jobs in  $\mathcal{J}_O$  are arranged in an optimal schedule  $\pi^* = (1, 2, \dots, n_O)$  ordered by the largest delivery time first (LDT) rule, i.e., job  $i$  is scheduled before job  $j$  if  $q_i \geq q_j$ , to minimize the maximum delivery completion time.

In this planned schedule  $\pi^*$ , the completion time  $C_j(\pi^*)$  for each original job  $j$ ,  $j = 1, 2, \dots, n_O$  is determined. During rescheduling, new jobs are inserted into the original schedule, potentially causing disruptions to the completion times of the original jobs. Let  $\sigma$  denote the new schedule after rescheduling, which may include both original and new jobs. The completion time of original job  $j$  in schedule  $\sigma$  is denoted as  $C_j(\sigma)$  for  $j = 1, 2, \dots, n_O$ . The disruption for each original job  $j$  is then defined as  $\Delta_j = |C_j(\sigma) - C_j(\pi^*)|$  for  $j = 1, 2, \dots, n_O$ . To ensure an acceptable service level, the maximum disruption  $\Delta_{\max} = \max_{j=1,2,\dots,n_O} \Delta_j (\leq T)$  must not exceed a given threshold  $T$ , where  $T$  represents the maximum allowable disruption.

Let  $\mathcal{J}_N^A$  and  $\mathcal{J}_N^R$  represent the sets of accepted and rejected new jobs, respectively. The rejected jobs do not need to be scheduled, but their rejection incurs a cost. The accepted new jobs in  $\mathcal{J}_N^A$  are inserted into the original schedule. The goal is to minimize the total cost, which includes the scheduling objective, the disruption penalty, and the rejection cost. The scheduling objective is to minimize the maximum delivery completion time  $D_{\max}$ , where  $D_{\max} = \max_{j \in \mathcal{J}_O \cup \mathcal{J}_N^A} \{C_j(\sigma) + q_j\}$ .

Using the well-known three-field notation introduced by Graham et al. [6], the problem can be expressed as

$$1|rej, \Delta_{\max} \leq T|D_{\max} + \mu\Delta_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j,$$

where  $\mu \geq 0$  represents the relative cost of one unit of disruption compared to the scheduling cost, and  $\sum_{j \in \mathcal{J}_N^R} e_j$  denotes the total rejection cost.

The problem encompasses the following three special cases:

(a) Ordinary scheduling problem with job rejection

$$1|rej|D_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j,$$

which arises when  $T$  is sufficiently large and  $\mu = 0$ .

(b) Constrained scheduling problem with job rejection

$$1|rej, \Delta_{\max} \leq T|D_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j,$$

which applies when  $\mu = 0$ .

(c) Total cost problem

$$1|rej|D_{\max} + \mu\Delta_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j,$$

which applies when  $T$  is sufficiently large.

### 3 Complexity Result

In this section, we demonstrate that the problem  $1|rej|D_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$  is NP-hard.

The decision problem of  $1|rej|D_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$  is described as follows:

Given a set of  $n (= n_O + n_N)$  independent jobs, consisting of the set of original jobs  $\mathcal{J}_O = \{1, 2, \dots, n_O\}$  and the set of new jobs  $\mathcal{J}_N = \{n_O + 1, n_O + 2, \dots, n_O + n_N\}$ , a single machine, and a threshold  $K$ . Each job  $j \in \mathcal{J}_O \cup \mathcal{J}_N$  has an associated processing time  $p_j$  and deliver time  $q_j$ . Additionally, each new job  $j \in \mathcal{J}_N$  has an associated rejection cost  $e_j$ . The decision problem asks whether there exists a schedule for a subset of jobs  $\mathcal{J}_O \cup \mathcal{J}_N \setminus \mathcal{J}_N^R$  on the machine such that  $D_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j \leq K$ .

We reduce the well-known Partition Problem [4] to this problem, thus proving that even with  $\mathcal{J}_O = \emptyset$ , the problem  $1|rej|D_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$  is NP-hard.

**Theorem 1** *Problem  $1|rej|D_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$  is NP-hard.*

**Proof** The problem  $1|rej|D_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$  is clearly in NP. To prove that it is NP-hard, we reduce the Partition Problem [4] to the problem  $1|rej|D_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$ . The Partition Problem is defined as follows:

Given a set  $A = \{a_1, a_2, \dots, a_n\}$  of  $n$  numbers such that  $\sum_{i=1}^n a_i = 2b$ , is there a subset  $A'$  of  $A$  such that  $\sum_{a_i \in A'} a_i = b$ ?

Given an instance  $A = \{a_1, a_2, \dots, a_n\}$  of the Partition Problem, we construct an instance of the problem  $1|rej|D_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$  with  $n+1$  jobs,  $1, 2, \dots, n, n+1$ . Here,  $1, 2, 3, \dots, n+1$  are the new jobs. Job 1 has processing time  $b$  and deliver time  $2b$  with rejection cost  $e_1 = +\infty$ . For  $i = 1, 2, \dots, n$ , each element  $a_i$  from the Partition Problem corresponds to a job  $i+1$  in the problem  $1|rej|D_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$  with processing time  $p_{i+1} = a_i$ , deliver time  $q_{i+1} = b$  and rejection cost  $e_{i+1} = a_i/2$ , where  $b = \frac{1}{2} \sum_{i=1}^n a_i$ . Set  $K = \frac{7}{2}b$ .

Consider any optimal schedule for the problem  $1|rej|D_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$ . Since job 1 is the new job with the largest deliver time and  $e_1 = +\infty$ , it must be accepted and scheduled first. Let  $\mathcal{J}_N^A$  and  $\mathcal{J}_N^R$  be the sets of indices of the scheduled and rejected jobs, respectively, among jobs  $2, 3, \dots, n+1$ . Let  $x = \sum_{i \in \mathcal{J}_N^A} p_i = \sum_{i \in \mathcal{J}_N^A} a_{i-1}$ . The makespan of the set of jobs in  $\{1\} \cup \mathcal{J}_N^A$  is  $b+x$ , and since all jobs in  $\mathcal{J}_N^A$  have the same deliver time  $b$ , the maximum deliver completion time is  $\max\{3b, 2b+x\}$  for jobs in  $\{1\} \cup \mathcal{J}_N^A$ . The total cost of the rejected jobs is  $\sum_{i \in \mathcal{J}_N^R} e_i = \sum_{i \in \mathcal{J}_N^R} a_{i-1}/2 = (2b-x)/2 = b-x/2$ . Therefore, the objective function for this schedule is  $\max\{2b+x, 3b\} + (b-x/2)$ .

This function has a unique minimum of  $\frac{7}{2}b$  when  $x = b$ . Hence, the optimum solution is achieved when  $\sum_{i \in \mathcal{J}_N^A} p_i = b$ , which corresponds to the solution of the Partition problem if it exists. Therefore, if the optimum solution to  $1|rej|D_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$  equals  $\frac{7}{2}b$ , then there exists a subset  $A' = \mathcal{J}_N^A$  of  $A$  such that  $\sum_{a_i \in A'} a_i = b$ , meaning the answer to the Partition Problem is “Yes”, and  $\mathcal{J}_N^A$  provides a valid partition. Conversely, if the optimum solution to  $1|rej|D_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$  is greater than  $\frac{7}{2}b$ , no such partition exists, and the answer to the Partition Problem is “No”. Similarly, if the answer to the Partition Problem is “Yes”, the optimum solution to  $1|rej|D_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$  is  $\frac{7}{2}b$ . If the answer to Partition Problem is “No”, the optimum solution is greater than  $\frac{7}{2}b$ .

**Remark** Since the problem  $1|rej|D_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$  is a special case of the problem  $1|rej, \Delta_{\max} \leq T|D_{\max} + \mu\Delta_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$ , we conclude that the problem  $1|rej, \Delta_{\max} \leq T|D_{\max} + \mu\Delta_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$  is NP-hard.

## 4 Structural Properties and Exact Algorithm

In this section, we describe the structural properties that hold for the optimal solution of the problem introduced in Sect. 2. These properties will guide the design of the exact algorithm.

For the problem  $1|rej, \Delta_{\max} \leq T|D_{\max} + \mu\Delta_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$ , we establish the following lemma, which describes the structure of an optimal schedule.

First, sequence the jobs in  $\mathcal{J}_O \cup \mathcal{J}_N$  in the largest deliver time first, indexed by  $1, 2, \dots, n$  with  $q_1 \geq q_2 \geq \dots \geq q_n$ , where the index order of the original jobs in  $\pi^*$  is preserved. Additionally, let  $l_o$  denote the last job in  $\mathcal{J}_O$  in the LDT order. We now establish the following structural properties:

**Lemma 1** *For the problem  $1|rej, \Delta_{\max} \leq T|D_{\max} + \mu\Delta_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$ , there exists an optimal schedule  $\sigma^*$  with the following properties:*

- The accepted jobs from  $\mathcal{J}_N$ , scheduled before the last job  $l_o$  in  $\mathcal{J}_O$ , and all the jobs in  $\mathcal{J}_O$ , follow their index order LDT;
- The accepted jobs from  $\mathcal{J}_N$ , scheduled after the last job  $l_o$  in  $\mathcal{J}_O$ , also follow their index order LDT;
- Any accepted job  $j \in \mathcal{J}_N$  with  $j > l_o$  is scheduled after the last job  $l_o$  in  $\mathcal{J}_O$ .

**Proof** Let  $\sigma$  denote an optimal schedule. Then,  $l_o$  is the last original job in  $\sigma$  in the LDT order. Let  $P_\Delta$  be the total processing time of the accepted new jobs scheduled before  $l_o$  in  $\sigma$ . Clearly, we have:  $P_\Delta = \Delta_{l_o}(\sigma) = \Delta_{\max}(\sigma) \leq T$ . We now construct a new schedule  $\sigma^*$  from  $\sigma$  by implementing the following two operations:

1. Reschedule job  $l_o$  and all jobs scheduled before  $l_o$  in  $\sigma$  in their index order to obtain an intermediate schedule  $\hat{\sigma}$ , and let  $\hat{l}_o$  be the last original job in  $\hat{\sigma}$ .
2. Reschedule all the accepted new jobs scheduled after job  $\hat{l}_o$  in  $\hat{\sigma}$  in their index order to obtain schedule  $\sigma^*$ .

For each job  $j \in \mathcal{J}_O$ ,  $\Delta_j(\sigma^*) (= \Delta_j(\hat{\sigma}))$  is equal to the total processing time of the accepted new jobs scheduled before job  $j$  in  $\hat{\sigma}$ . Then,  $\Delta_{\max}(\sigma^*) = \Delta_{\max}(\hat{\sigma}) \leq P_\Delta = \Delta_{\max}(\sigma) \leq T$ , which means that both  $\sigma^*$  and  $\hat{\sigma}$  are feasible. Since LDT order is optimal for minimizing  $D_{\max}$ , (due to the sequencing rule  $q_1 \geq q_2 \geq \dots \geq q_n$ ), we can deduce that  $\hat{\sigma}$  is at least as good as  $\sigma$ , and  $\sigma^*$  is at least as good as  $\hat{\sigma}$ . Thus,  $\sigma^*$  is an optimal schedule that satisfies the properties described in Lemma 1.

Using the structural properties established in Lemma 1, we can derive a dynamic programming (DP) exact algorithm to solve the problem  $1|rej, \Delta_{\max} \leq T|D_{\max} + \mu\Delta_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$ . To facilitate the description, we partition the schedule into two parts:

- The **left schedule** consists of the jobs scheduled before job  $l_o$ , including  $l_o$ .
- The **right schedule** consists of the jobs scheduled after job  $l_o$ .

We use a value function

$$E(j; C_j^b, \Delta_j, t_j^a, D_j^b, D_j^a)$$

to represent the minimum total rejection cost of rejected new jobs for a schedule of jobs  $1, 2, \dots, j$ , where

- $j$ : Number of jobs considered so far.
- $C_j^b$ : Completion time before job  $l_o$  in the left schedule.

- $\Delta_j$ : Total processing time of the accepted new jobs before  $l_o$ .
- $t_j^a$ : Total processing time of accepted new jobs after  $l_o$  in the right schedule.
- $D_j^b$ : Maximum deliver completion time of jobs before  $l_o$  in the left schedule.
- $D_j^a$ : Maximum deliver completion time of jobs after  $l_o$  in the right schedule, where the starting time in the right schedule is initially set to 0.

The initial condition

$$E(0; C_0^b, \Delta_0, t_0^a, D_0^b, D_0^a) = E(0; 0, 0, 0, 0, 0) = 0,$$

which represents the empty initial schedule, where no jobs have been scheduled and no rejection cost has been incurred.

General speaking, given a value function  $E(j; C_j^b, \Delta_j, t_j^a, D_j^b, D_j^a)$ , it leads to at most four value functions with the form  $E(j+1; C_{j+1}^b, \Delta_{j+1}, t_{j+1}^a, D_{j+1}^b, D_{j+1}^a)$ . We consider the following cases.

**Case (a)** If  $j+1 \in \mathcal{J}_O$ , schedule job  $j+1$  at the end in the left schedule and we thus achieve a partial schedule on the first  $j+1$  jobs in which the current completion time in the left schedule is  $C_{j+1}^b = C_j^b + p_{j+1}$ , the current total processing time of jobs before job  $l_o$  in  $\mathcal{J}_N$  is  $\Delta_{j+1} = \Delta_j$ , the current total processing time of jobs after job  $l_o$  in  $\mathcal{J}_N$  in the right schedule is  $t_{j+1}^a = t_j^a$ , the current maximum deliver completion time before job  $l_o$  (including  $l_o$ ) in the left schedule is  $D_{j+1}^b = \max\{D_j^b, C_{j+1}^b + q_{j+1}\}$ , the current maximum deliver completion time of jobs after job  $l_o$  in the right schedule is  $D_{j+1}^a = D_j^a$  where the starting time in the right schedule is initially set to 0. We summarize recurrence on the value function in **Case (a)** as follows:

$$\begin{aligned} E(j+1; C_{j+1}^b, \Delta_{j+1}, t_{j+1}^a, D_{j+1}^b, D_{j+1}^a) \\ = E(j; C_j^b + p_{j+1}, \Delta_j, t_j^a, \max\{D_j^b, C_{j+1}^b + q_{j+1}\}, D_j^a). \end{aligned} \quad (1)$$

**Case (b)** If  $j+1 \in \mathcal{J}_N$  and  $j+1 < l_o$  with  $\Delta_j + p_{j+1} \leq T$ , accept job  $j+1$  and schedule it at the end in the left schedule. We thus achieve a partial schedule on the first  $j+1$  jobs in which the current completion time in the left schedule is  $C_{j+1}^b = C_j^b + p_{j+1}$ , the current total processing time of jobs in  $\mathcal{J}_N$  before job  $l_o$  is  $\Delta_{j+1} = \Delta_j + p_{j+1}$ , the current total processing time of jobs in  $\mathcal{J}_N$  after job  $l_o$  in the right schedule is  $t_{j+1}^a = t_j^a$ , the current maximum deliver time before job  $l_o$  (including  $l_o$ ) in the left schedule is  $D_{j+1}^b = \max\{D_j^b, C_{j+1}^b + q_{j+1}\}$ , the current maximum deliver completion time of jobs after job  $l_o$  in the right schedule is  $D_{j+1}^a = D_j^a$  where the starting time in the right schedule is initially set to 0. We summarize the recurrence on the value function in **Case (b)** as follows:

$$\begin{aligned} E(j+1; C_{j+1}^b, \Delta_{j+1}, t_{j+1}^a, D_{j+1}^b, D_{j+1}^a) \\ = E(j; C_j^b + p_{j+1}, \Delta_j + p_{j+1}, t_j^a, \max\{D_j^b, C_{j+1}^b + q_{j+1}\}, D_j^a). \end{aligned} \quad (2)$$



**Case (c)** If  $j + 1 \in \mathcal{J}_N$ , accept job  $j + 1$  and schedule it at the end in the right schedule. We thus achieve a partial schedule on the first  $j + 1$  jobs in which the current completion time in the left schedule is  $C_{j+1}^b = C_j^b$ , the current total processing time of jobs in  $\mathcal{J}_N$  before job  $l_o$  is  $\Delta_{j+1} = \Delta_j$ , the current total processing time of jobs in  $\mathcal{J}_N$  after job  $l_o$  in the right schedule is  $t_{j+1}^a = t_j^a + p_{j+1}$ , the current maximum deliver completion time before job  $l_o$  (including  $l_o$ ) in the left schedule is  $D_{j+1}^b = D_j^b$ , the current maximum deliver completion time of jobs after job  $l_o$  in the right schedule is  $D_{j+1}^a = \max\{D_j^a, t_j^a + p_{j+1} + q_{j+1}\}$  where the starting time in the right schedule is initially set to 0. We summarize the recurrence on the value function in **Case (c)** as follows:

$$\begin{aligned} E(j+1; C_{j+1}^b, \Delta_{j+1}, t_{j+1}^a, D_{j+1}^b, D_{j+1}^a) \\ = E(j; C_j^b, \Delta_j, t_j^a + p_{j+1}, D_j^b, \max\{D_j^a, t_j^a + p_{j+1} + q_{j+1}\}). \end{aligned} \quad (3)$$

**Case (d)** If  $j + 1 \in \mathcal{J}_N$ , reject job  $J_{j+1}$ . We thus achieve a partial schedule on the first  $j + 1$  jobs in which the current completion time in the left schedule is  $C_{j+1}^b = C_j^b$ , the current total processing time of jobs in  $\mathcal{J}_N$  before job  $J_{l_o}$  is  $\Delta_{j+1} = \Delta_j$ , the current total processing time of jobs in  $\mathcal{J}_N$  after job  $J_{l_o}$  in the right schedule is  $t_{j+1}^a = t_j^a$ , the current maximum deliver completion time before job  $J_{l_o}$  (including  $J_{l_o}$ ) in the left schedule is  $D_{j+1}^b = D_j^b$ , the current maximum deliver completion time of jobs after job  $J_{l_o}$  in the right schedule is  $D_{j+1}^a = D_j^a$  where the starting time in the right schedule is initially set to 0. We summarize the recurrence on the value function in **Case (d)** as follows:

$$E(j+1; C_{j+1}^b, \Delta_{j+1}, t_{j+1}^a, D_{j+1}^b, D_{j+1}^a) = E(j; C_j^b, \Delta_j, t_j^a, D_j^b, D_j^a) + e_{j+1}. \quad (4)$$

For each value function  $E(j+1; C_{j+1}^b, \Delta_{j+1}, t_{j+1}^a, D_{j+1}^b, D_{j+1}^a)$  with the same values on the variables  $C_{j+1}^b, \Delta_{j+1}, t_{j+1}^a, D_{j+1}^b$ , and  $D_{j+1}^a$ , we retain only one with the minimum total rejection cost of rejected new jobs.

Now, we present Algorithm 1 to solve problem 1|rej,  $\Delta_{\max} \leq T$ | $D_{\max} + \mu \Delta_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$  as follows:

**Algorithm 1** Exact algorithm for problem  $1|rej, \Delta_{\max} \leq T|D_{\max} + \mu\Delta_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$

- 1 Sort all the jobs in  $\mathcal{J}_O \cup \mathcal{J}_N$  in LDT order, i.e.,  $q_1 \geq q_2 \geq \dots \geq q_n$ . Let the last job belonging to  $\mathcal{J}_O$  in jobs  $\{1, 2, \dots, n\}$  be  $l_o$ .
- 2 Set all  $E(j; C_j^b, \Delta_j, t_j^a, D_j^b, D_j^a) = +\infty$  except  $E(0; 0, 0, 0, 0, 0) = 0$ , where  $0 \leq j \leq n$ ,  $0 \leq C_j^b \leq np_{\max}$ ,  $0 \leq \Delta_j \leq T$ ,  $0 \leq t_j^a \leq n_N p_{\max}$ ,  $0 \leq D_j^b \leq np_{\max} + q_{\max}$ , and  $0 \leq D_j^a \leq n_N p_{\max} + q_{\max}$  with  $p_{\max} = \max_{j=1,2,\dots,n} p_j$ , and  $q_{\max} = \max_{j=1,2,\dots,n} q_j$ .
- 3 Starting from the initial value function  $E(0; 0, 0, 0, 0, 0) = 0$ , for each value function  $E(j; C_j^b, \Delta_j, t_j^a, D_j^b, D_j^a)$ , set all the value functions of the form  $E(j+1; C_{j+1}^b, \Delta_{j+1}, t_{j+1}^a, D_{j+1}^b, D_{j+1}^a)$  by using the recurrences as shown in Eqs. (1), (2), (3), and (4). For each value function  $E(j+1; C_{j+1}^b, \Delta_{j+1}, t_{j+1}^a, D_{j+1}^b, D_{j+1}^a)$  with the same values on the variables  $C_{j+1}^b, \Delta_{j+1}, t_{j+1}^a, D_{j+1}^b$ , and  $D_{j+1}^a$ , retain only one with the minimum total rejection cost of rejected new jobs.
- 4 Among all the value functions of the form  $E(n; C_n^b, \Delta_n, t_n^a, D_n^b, D_n^a)$ , find the one with the minimum value of  $\max\{D_n^b, C_n^b + D_n^a\} + \mu\Delta_n + E(n; C_n^b, \Delta_n, t_n^a, D_n^b, D_n^a)$ , and by backtracking, determine the corresponding subset of accepted jobs from  $\mathcal{J}_N$  before job  $l_o$ , and the subset of accepted jobs from  $\mathcal{J}_N$  after job  $l_o$ .
- 5 Schedule the above jobs according to the rules described in Lemma 1.

In the end, we also obtain the following theorem.

**Theorem 2** Algorithm 1 finds an optimal solution for problem  $1|rej, \Delta_{\max} \leq T|D_{\max} + \mu\Delta_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$  in  $O(n^2 n_N p_{\max}^2 (np_{\max} + q_{\max})(n_N p_{\max} + q_{\max})T)$  time.

**Proof** Since Algorithm 1 leverages the optimal structural properties as justified in Lemma 1 and compares the overall cost of all possible value function transitions, it generates an optimal solution. The running time is  $O(n^2 n_N p_{\max}^2 (np_{\max} + q_{\max})(n_N p_{\max} + q_{\max})T)$ .

## 5 An FPTAS for Problem $1|rej, \Delta_{\max} \leq T|D_{\max} + \mu\Delta_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$

In this section, we derive a fully polynomial-time approximation scheme (FPTAS) for the problem  $1|rej, \Delta_{\max} \leq T|D_{\max} + \mu\Delta_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$ , building upon the pseudo-polynomial-time dynamic programming exact algorithm (Algorithm 1) presented in Section 4 with some minor changes, by employing a trimming technique.

We use a 7-dimensional vector  $(j; C_j^b, \Delta_j, t_j^a, D_j^b, D_j^a, E_j)$  to describe a state, where  $0 \leq j \leq n$ ,  $0 \leq C_j^b \leq np_{\max}$ ,  $0 \leq \Delta_j \leq T$ ,  $0 \leq t_j^a \leq n_N p_{\max}$ ,  $0 \leq D_j^b \leq np_{\max} + q_{\max}$ ,  $0 \leq D_j^a \leq n_N p_{\max} + q_{\max}$ , and  $0 \leq E_j \leq n_N e_{\max}$ .

The approximation scheme, denoted as Algorithm 2, is presented below.

First, define  $\delta = (1 + \varepsilon/2n)$  and set

$$v_1 = \lceil \log_{\delta}^{np_{\max}} \rceil, v_2 = \lceil \log_{\delta}^T \rceil, v_3 = \lceil \log_{\delta}^{n_N p_{\max}} \rceil, v_4 = \lceil \log_{\delta}^{np_{\max} + q_{\max}} \rceil, v_5 = \lceil \log_{\delta}^{n_N p_{\max} + q_{\max}} \rceil, \text{ and } v_6 = \lceil \log_{\delta}^{n_N e_{\max}} \rceil.$$

Next, split the range of the second dimension  $[0, np_{\max}]$  into

$$I_0^{C^b} = [0, 1), I_i^{C^b} = [\delta^{i-1}, \delta^i), 1 \leq i \leq v_1 - 1, I_{v_1}^{C^b} = [\delta^{v_1-1}, np_{\max}];$$

split the third dimension  $[0, T]$  into

$$I_0^\Delta = [0, 1), I_i^\Delta = [\delta^{i-1}, \delta^i), 1 \leq i \leq v_2 - 1, I_{v_2}^\Delta = [\delta^{v_2-1}, T];$$

split the fourth dimension  $[0, n_N p_{\max}]$  into

$$I_0^{t^a} = [0, 1), I_i^{t^a} = [\delta^{i-1}, \delta^i), 1 \leq i \leq v_3 - 1, I_{v_3}^{t^a} = [\delta^{v_3-1}, n_N p_{\max}];$$

split the fifth dimension  $[0, np_{\max} + q_{\max}]$  into

$$I_0^{D^b} = [0, 1), I_i^{D^b} = [\delta^{i-1}, \delta^i), 1 \leq i \leq v_4 - 1, I_{v_4}^{D^b} = [\delta^{v_4-1}, np_{\max} + q_{\max}];$$

and split the sixth dimension  $[0, n_N p_{\max} + q_{\max}]$  into

$$I_0^{D^a} = [0, 1), I_i^{D^a} = [\delta^{i-1}, \delta^i), 1 \leq i \leq v_5 - 1, I_{v_5}^{D^a} = [\delta^{v_5-1}, n_N p_{\max} + q_{\max}];$$

and split the seventh dimension  $[0, n_N e_{\max}]$  into

$$I_0^E = [0, 1), I_i^E = [\delta^{i-1}, \delta^i), 1 \leq i \leq v_6 - 1, I_{v_6}^E = [\delta^{v_6-1}, n_N e_{\max}].$$

Let  $[n] = \{0, 1, 2, \dots, n\}$  for any positive integer  $n$ .

Given a 6-dimensional vector

$$(r_1, r_2, r_3, r_4, r_5, r_6) \in [v_1] \times [v_2] \times [v_3] \times [v_4] \times [v_5] \times [v_6],$$

a box  $B(j; r_1, r_2, r_3, r_4, r_5, r_6)$  is associated with the collection of the states  $(j; C_j^b, \Delta_j, t_j^a, D_j^b, D_j^a, E_j)$  such that  $C_j^b \in I_{r_1}^{C^b}$ ,  $\Delta_j \in I_{r_2}^\Delta$ ,  $t_j^a \in I_{r_3}^{t^a}$ ,  $D_j^b \in I_{r_4}^{D^b}$ ,  $D_j^a \in I_{r_5}^{D^a}$ , and  $E_j \in I_{r_6}^E$ . Algorithm 2 operates similarly to the dynamic programming approach in Sect. 4, but uses a significantly smaller 7-dimensional Boolean table  $\text{DP}(j; r_1, r_2, r_3, r_4, r_5, r_6)$  for computation, where  $1 \leq j \leq n$  and  $(r_1, r_2, r_3, r_4, r_5, r_6) \in [v_1] \times [v_2] \times [v_3] \times [v_4] \times [v_5] \times [v_6]$ . The entry  $\text{DP}(j; r_1, r_2, r_3, r_4, r_5, r_6)$  is true if and only if there is a partial schedule for the first  $j$  jobs that satisfies the following:

- The completion time in the left schedule falls in  $I_{r_1}^{C^b}$ ,
- The maximum completion time disruption falls in  $I_{r_2}^\Delta$ ,
- The total processing time of jobs from  $\mathcal{J}_N$  before job  $l_o$  falls in  $I_{r_3}^{t^a}$ ,
- The current maximum delivery completion time of jobs before job  $l_o$  (including  $l_o$ ) in the left schedule falls in  $I_{r_4}^{D^b}$ ,
- The current maximum delivery completion time of jobs after job  $l_o$  in the right schedule (starting at time 0) falls in  $I_{r_5}^{D^a}$ ,

- The total rejection cost falls in  $I_{r_6}^E$ .

For each DP entry, the associated partial schedule with the least  $\Delta_j$  is retained, and all other partial schedules evaluating the same DP entry to true are discarded. Ties are broken arbitrarily.

For initialization, the entry  $DP(0; 0, 0, 0, 0, 0, 0)$  is set to true and the associated partial schedule is  $(0; 0, 0, 0, 0, 0, 0)$ . All other entries  $DP(0; r_1, r_2, r_3, r_4, r_5, r_6)$  except  $(0; 0, 0, 0, 0, 0, 0)$  are initialized to false for all  $(r_1, r_2, r_3, r_4, r_5, r_6) \in [v_1] \times [v_2] \times [v_3] \times [v_4] \times [v_5] \times [v_6]$ .

Given a true entry  $DP(j; r_1, r_2, r_3, r_4, r_5, r_6)$  and its associated partial schedule  $(j; C_j^b, \Delta_j, t_j^a, D_j^b, D_j^a, E_j)$ , at most four partial schedules  $(j+1; C_{j+1}^b, \Delta_{j+1}, t_{j+1}^a, D_{j+1}^b, D_{j+1}^a, E_{j+1})$  can be derived using the recurrence relations from Eqs. (5), (6), (7), and (8), described as

$$\begin{aligned} C_{j+1}^b &= C_j^b + p_{j+1}, \Delta_{j+1} = \Delta_j, t_{j+1}^a = t_j^a, \\ D_{j+1}^b &= \max\{D_j^b, C_{j+1}^b + q_{j+1}\}, D_{j+1}^a = D_j^a, E_{j+1} = E_j, \end{aligned} \quad (5)$$

$$\begin{aligned} C_{j+1}^b &= C_j^b + p_{j+1}, \Delta_{j+1} = \Delta_j + p_{j+1}, t_{j+1}^a = t_j^a, \\ D_{j+1}^b &= \max\{D_j^b, C_{j+1}^b + q_{j+1}\}, D_{j+1}^a = D_j^a, E_{j+1} = E_j, \end{aligned} \quad (6)$$

$$\begin{aligned} C_{j+1}^b &= C_j^b, \Delta_{j+1} = \Delta_j, t_{j+1}^a = t_j^a + p_{j+1}, \\ D_{j+1}^b &= D_j^b, D_{j+1}^a = \max\{D_j^a, t_j^a + p_{j+1} + q_{j+1}\}, E_{j+1} = E_j, \end{aligned} \quad (7)$$

and

$$\begin{aligned} C_{j+1}^b &= C_j^b, \Delta_{j+1} = \Delta_j, t_{j+1}^a = t_j^a, \\ D_{j+1}^b &= D_j^b, D_{j+1}^a = D_j^a, E_{j+1} = E_j + e_{j+1}, \end{aligned} \quad (8)$$

which correspond to cases (a), (b), (c), and (d), respectively.

When  $C_{j+1}^b(\Delta_{j+1}, t_{j+1}^a, D_{j+1}^b, D_{j+1}^a, E_{j+1})$ , respectively) belongs to the interval  $I_{r_1}^{C^b}(I_{r_2}^{\Delta}, I_{r_3}^{t^a}, I_{r_4}^{D^b}, I_{r_5}^{D^a}, I_{r_6}^E)$ , respectively) such that the derived partial schedule  $(j+1; C_{j+1}^b, \Delta_{j+1}, t_{j+1}^a, D_{j+1}^b, D_{j+1}^a, E_{j+1})$  evaluates the entry  $DP(j+1; r_1, r_2, r_3, r_4, r_5, r_6)$  to true, the following process is applied:

- If the entry  $DP(j+1; r_1, r_2, r_3, r_4, r_5, r_6)$  was previously false, then the partial schedule  $(j; C_j^b, \Delta_j, t_j^a, D_j^b, D_j^a, E_j)$  associated with the true entry  $DP(j; r_1, r_2, r_3, r_4, r_5, r_6)$  is referred to as the predecessor of the newly created partial schedule  $(j+1; C_{j+1}^b, \Delta_{j+1}, t_{j+1}^a, D_{j+1}^b, D_{j+1}^a, E_{j+1})$ .
- Otherwise, the disruption value  $\Delta_{j+1}$  of the new partial schedule is compared to the maximum completion time disruption of the saved partial schedule associated with the true entry  $DP(j+1; r_1, r_2, r_3, r_4, r_5, r_6)$ . If the new partial schedule has a smaller disruption value, it is saved and the predecessor information is correspondingly updated. If the previously saved schedule has a smaller or equal disruption, no update is made.

A key property of this tabular computation is that, for each saved partial schedule for the first  $j+1$  jobs, its predecessor partial schedule for the first  $j$  jobs is also saved.

At the end of filling the DP-table, for each 6-dimensional vector  $(r_1, r_2, r_3, r_4, r_5, r_6) \in [v_1] \times [v_2] \times [v_3] \times [v_4] \times [v_5] \times [v_6]$  such that  $\text{DP}(n; r_1, r_2, r_3, r_4, r_5, r_6)$  is true, we examine the associated schedule  $(n; C_n^b, \Delta_n, t_n^a, D_n^b, D_n^a, E_n)$ . Algorithm 2 outputs the one with the minimum value of  $\max\{D_n^b, C_n^b + D_n^a\} + \mu\Delta_n + E_n$ , denoted as  $(n; C_n^{b\varepsilon}, \Delta_n^\varepsilon, t_n^{a\varepsilon}, D_n^{b\varepsilon}, D_n^{a\varepsilon}, E_n^\varepsilon)$ .

We summarize Algorithm 2 as follows:

---

**Algorithm 2** An FPTAS for problem  $1|rej, \Delta_{\max} \leq T|D_{\max} + \mu\Delta_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$

---

- 1 Initialization:  $B(0; 0, 0, 0, 0, 0, 0)$  is true, and the corresponding state is  $(0; 0, 0, 0, 0, 0, 0)$ .
  - 2 **for**  $j = 0$  to  $n - 1$  **do**
  - 3   **for** any state  $s = (j; C_j^b, \Delta_j, t_j^a, D_j^b, D_j^a, E_j)$  such that  $B(j; r_1, r_2, r_3, r_4, r_5, r_6)$  is true for some  $(r_1, r_2, r_3, r_4, r_5, r_6) \in [v_1] \times [v_2] \times [v_3] \times [v_4] \times [v_5] \times [v_6]$  **do**
  - 4     Use the recurrences as shown in Eqs.(5), (6), (7), and (8) to derive the possible true state  $\tilde{s} = (j + 1; \tilde{C}_{j+1}^b, \tilde{\Delta}_{j+1}, \tilde{t}_{j+1}^a, \tilde{D}_{j+1}^b, \tilde{D}_{j+1}^a, \tilde{E}_{j+1})$  and determine the corresponding vector  $(r_1, r_2, r_3, r_4, r_5, r_6)$  such that  $\tilde{s} \in B(j + 1; r_1, r_2, r_3, r_4, r_5, r_6)$ .
  - 5     **if**  $B(j + 1; r_1, r_2, r_3, r_4, r_5, r_6)$  is false **then**
  - 6       Set  $\tilde{s}$  as the associated state of  $B(j + 1; r_1, r_2, r_3, r_4, r_5, r_6)$ . Denote the state  $s$  associated with  $B(j; r_1, r_2, r_3, r_4, r_5, r_6)$  as the predecessor of the state  $\tilde{s}$  associated with  $B(j + 1; r_1, r_2, r_3, r_4, r_5, r_6)$ .
  - 7     **else**
  - 8       Suppose its previous associated state is  $\hat{s} = (j + 1; \hat{C}_{j+1}^b, \hat{\Delta}_{j+1}, \hat{t}_{j+1}^a, \hat{D}_{j+1}^b, \hat{D}_{j+1}^a, \hat{E}_{j+1})$ .
  - 9       Compare  $\tilde{s}$  and  $\hat{s}$ , and choose the one with smaller value of  $\Delta_j$  and update it as the associated state of  $B(j + 1; r_1, r_2, r_3, r_4, r_5, r_6)$ . Update the predecessor relationship accordingly.
  - 10 From all the  $(r_1, r_2, r_3, r_4, r_5, r_6) \in [v_1] \times [v_2] \times [v_3] \times [v_4] \times [v_5] \times [v_6]$  such that  $B(n; r_1, r_2, r_3, r_4, r_5, r_6)$  is true, choose the state that with minimum value of  $\max\{D_n^b, C_n^b + D_n^a\} + \mu\Delta_n + E_n$ , and denote this state as  $(n; C_n^{b\varepsilon}, \Delta_n^\varepsilon, t_n^{a\varepsilon}, D_n^{b\varepsilon}, D_n^{a\varepsilon}, E_n^\varepsilon)$ .
- 

The following theorem summarizes the approximability result achieved for problem  $1|rej, \Delta_{\max} \leq \Delta|D_{\max} + \mu\Delta_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$ .

**Theorem 3** For any  $\varepsilon > 0$ , the problem  $1|rej, \Delta_{\max} \leq T|D_{\max} + \mu\Delta_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$  admits an  $O(n^6(\log M)^5/\varepsilon^5)$ -time approximation algorithm achieving a performance ratio of  $(1 + \varepsilon)$ , where  $M = \max\{p_{\max}, q_{\max}, e_{\max}\}$ .

**Proof** The proof of the performance ratio is done by induction. Assume that

$$(1; C_1^{b*}, \Delta_1^*, t_1^{a*}, D_1^{b*}, D_1^{a*}, E_1^*) \rightarrow (2; C_2^{b*}, \Delta_2^*, t_2^{a*}, D_2^{b*}, D_2^{a*}, E_2^*) \rightarrow \dots \rightarrow (n; C_n^{b*}, \Delta_n^*, t_n^{a*}, D_n^{b*}, D_n^{a*}, E_n^*)$$

is the sequence of 7-dimensional vectors computed by the exact algorithm (Algorithm 1) leading to the optimal solution for problem  $1|rej, \Delta_{\max} \leq T|D_{\max} + \mu\Delta_{\max} + \sum_{j \in \mathcal{J}_N^R} e_j$ . The induction statement is as follows: for each  $1 \leq j \leq n - 1$ , there exists a 7-dimensional vector  $(j; C_j^b, \Delta_j, t_j^a, D_j^b, D_j^a, E_j)$  saved by the Algorithm 2, such that  $C_j^b \leq C_j^{b*}, \Delta_j \leq \Delta_j^*, t_j^a \leq t_j^{a*}, D_j^b \leq D_j^{b*}, D_j^a \leq D_j^{a*}$ , and  $E_j \leq E_j^*$ .

The base case occurs at  $j = 1$ , and the statement holds since there are at most four possible partial schedules for the first job, described as  $(1; p_1, 0, 0, p_1 + q_1, 0, 0)$ ,  $(1; p_1, p_1, 0, p_1 + q_1, 0, 0)$ ,  $(1; 0, 0, p_1, 0, p_1 + q_1, 0)$ , or  $(1; 0, 0, 0, 0, 0, e_1)$ .

We assume that the induction statement holds for  $j$ , where  $1 \leq j \leq n-1$ . That is, there exists a 7-dimensional vector  $(j; C_j^b, \Delta_j, t_j^a, D_j^b, D_j^a, E_j)$  saved by Algorithm 2, such that  $C_j^b \leq C_j^{b*} \delta^j$ ,  $\Delta_j \leq \Delta_j^*$ ,  $t_j^a \leq t_j^{a*} \delta^j$ ,  $D_j^b \leq D_j^{b*} \delta^j$ ,  $D_j^a \leq D_j^{a*} \delta^j$ , and  $E_j \leq E_j^* \delta^j$ . We now consider job  $j+1$ .

In **Case (a)**, job  $j+1$  belongs to  $\mathcal{J}_O$  and is added to the end of the left schedule, resulting in a partial schedule as described in Eq. (5). Thus, we have:  $C_{j+1}^b = C_j^b + p_{j+1}$ ,  $\Delta_{j+1} = \Delta_j$ ,  $t_{j+1}^a = t_j^a$ ,  $D_{j+1}^b = \max\{D_j^b, C_{j+1}^b + d_{j+1}\}$ ,  $D_{j+1}^a = D_j^a$ , and  $E_{j+1} = E_j$ .

Assume this 7-dimensional vector falls in the box  $(j+1)-B_{r_1, r_2, r_3, r_4, r_5, r_6}$ , then the 7-dimensional vector  $(j+1; \hat{C}_{j+1}^b, \hat{\Delta}_{j+1}, \hat{t}_{j+1}^a, \hat{D}_{j+1}^b, \hat{D}_{j+1}^a, \hat{E}_{j+1})$  saved in this box by Algorithm 2 must satisfy  $\hat{C}_{j+1}^b \leq C_{j+1}^b \delta$ ,  $\hat{\Delta}_{j+1} \leq \Delta_{j+1}$ ,  $\hat{t}_{j+1}^a \leq t_{j+1}^a \delta$ ,  $\hat{D}_{j+1}^b \leq D_{j+1}^b \delta$ ,  $\hat{D}_{j+1}^a \leq D_{j+1}^a \delta$ , and  $\hat{E}_{j+1} \leq E_{j+1} \delta$ .

If  $(j; C_j^{b*}, \Delta_j^*, t_j^{b*}, D_j^{b*}, D_j^{a*}, E_j^*)$  leads to  $(j+1; C_{j+1}^{b*}, \Delta_{j+1}^*, t_{j+1}^{b*}, D_{j+1}^{b*}, D_{j+1}^{a*}, E_{j+1}^*)$  by adding job  $j+1$  to the end in the left schedule, then we have  $C_{j+1}^{b*} = C_j^{b*} + p_{j+1}$ ,  $\Delta_{j+1}^* = \Delta_j^*$ ,  $t_{j+1}^{a*} = t_j^{a*}$ ,  $D_{j+1}^{b*} = \max\{D_j^{b*}, C_{j+1}^{b*} + d_{j+1}\}$ ,  $D_{j+1}^{a*} = D_j^{a*}$  and  $E_{j+1}^* = E_j^*$ .

From Eqs. (5), we have the following recurrence relations:  $C_{j+1}^b = C_j^b + p_{j+1}$ ,  $\Delta_{j+1} = \Delta_j$ ,  $t_{j+1}^a = t_j^a$ ,  $D_{j+1}^b = \max\{D_j^b, C_{j+1}^b + d_{j+1}\}$ ,  $D_{j+1}^a = D_j^a$ , and  $E_{j+1} = E_j$ . With  $\delta > 1$ , we can bound the corresponding terms as follows:

$$\begin{aligned} \hat{C}_{j+1}^b &\leq C_{j+1}^b \delta \leq (C_j^b + p_{j+1}) \delta \leq (C_j^{b*} \delta^j + p_{j+1}) \delta \leq (C_j^{b*} + p_{j+1}) \delta^{j+1} \leq C_{j+1}^{b*} \delta^{j+1}, \\ \hat{\Delta}_{j+1} &\leq \Delta_{j+1} = \Delta_j \leq \Delta_j^* \leq \Delta_{j+1}^*, \\ \hat{t}_{j+1}^a &\leq t_{j+1}^a \delta = t_j^a \delta \leq t_j^{a*} \delta^{j+1} = t_{j+1}^{a*} \delta^{j+1}, \\ \hat{D}_{j+1}^b &\leq D_{j+1}^b \delta = \max\{D_j^b, C_{j+1}^b + q_{j+1}\} \delta \leq \max\{D_j^{b*} \delta^j, C_{j+1}^{b*} \delta^{j+1} + q_{j+1}\} \\ &\leq \max\{D_j^{b*}, C_{j+1}^{b*} + q_{j+1}\} \delta^{j+1} = D_{j+1}^{b*} \delta^{j+1}, \\ \hat{D}_{j+1}^a &\leq D_{j+1}^a \delta = D_j^a \delta \leq D_j^{a*} \delta^{j+1} = D_{j+1}^{a*} \delta^{j+1}, \\ \hat{E}_{j+1} &\leq E_{j+1} \delta \leq E_j^* \delta^{j+1} \leq E_{j+1}^* \delta^{j+1}. \end{aligned}$$

In **Case (b)**, job  $j+1$  is from the set of new jobs  $\mathcal{J}_N$ , and  $j+1 \leq j_o$ . It is added to the left schedule, resulting in a partial schedule as described in Eqs. (6). Here, we have the following recurrence relations:  $C_{j+1}^b = C_j^b + p_{j+1}$ ,  $\Delta_{j+1} = \Delta_j + p_{j+1}$ ,  $t_{j+1}^a = t_j^a$ ,  $D_{j+1}^b = \max\{D_j^b, C_{j+1}^b + q_{j+1}\}$ ,  $D_{j+1}^a = D_j^a$ , and  $E_{j+1} = E_j$ .

Now, assume this 7-dimensional vector falls within the box  $(j+1)-B_{r_1, r_2, r_3, r_4, r_5, r_6}$ . Then, the 7-dimensional vector  $(j+1; \hat{C}_{j+1}^b, \hat{\Delta}_{j+1}, \hat{t}_{j+1}^a, \hat{D}_{j+1}^b, \hat{D}_{j+1}^a, \hat{E}_{j+1})$  saved in this box by Algorithm 2 must satisfy  $\hat{C}_{j+1}^b \leq C_{j+1}^b \delta$ ,  $\hat{\Delta}_{j+1} \leq \Delta_{j+1}$ ,  $\hat{t}_{j+1}^a \leq t_{j+1}^a \delta$ ,  $\hat{D}_{j+1}^b \leq D_{j+1}^b \delta$ ,  $\hat{D}_{j+1}^a \leq D_{j+1}^a \delta$ , and  $\hat{E}_{j+1} \leq E_{j+1} \delta$ .

If  $(j; C_j^{b*}, \Delta_j^*, t_j^{b*}, D_j^{b*}, D_j^{a*}, E_j^*)$  leads to  $(j+1; C_{j+1}^{b*}, \Delta_{j+1}^*, t_{j+1}^{b*}, D_{j+1}^{b*}, D_{j+1}^{a*}, E_{j+1}^*)$  also by appending job  $j+1$  in the left schedule, we have  $C_{j+1}^{b*} = C_j^{b*} + p_{j+1}$ ,  $\Delta_{j+1}^* = \Delta_j^* + p_{j+1}$ ,  $t_{j+1}^{a*} = t_j^{a*}$ ,  $D_{j+1}^{b*} = \max\{D_j^{b*}, C_{j+1}^{b*} + q_{j+1}\}$ ,  $D_{j+1}^{a*} = D_j^{a*}$ , and  $E_{j+1}^* = E_j^*$ .

From Eqs. (6), we obtain:  $C_{j+1}^b = C_j^b + p_{j+1}$ ,  $\Delta_{j+1} = \Delta_j + p_{j+1}$ ,  $t_{j+1}^a = t_j^a$ ,  $D_{j+1}^b = \max\{D_j^b, C_{j+1}^b + q_{j+1}\}$ ,  $D_{j+1}^a = D_j^a$ , and  $E_{j+1} = E_j$ . With  $\delta > 1$ , we can now bound the terms

$$\begin{aligned}\hat{C}_{j+1}^b &\leq C_{j+1}^b \delta \leq (C_j^b + p_{j+1})\delta \leq (C_j^{b*} \delta^j + p_{j+1})\delta \leq (C_j^{b*} + p_{j+1})\delta^{j+1} \leq C_{j+1}^{b*} \delta^{j+1}, \\ \hat{\Delta}_{j+1} &\leq \Delta_{j+1} = \Delta_j + p_{j+1} \leq \Delta_j^* + p_{j+1} \leq \Delta_{j+1}^*, \\ \hat{t}_{j+1}^a &\leq t_{j+1}^a \delta = t_j^a \delta \leq t_j^{a*} \delta^{j+1} = t_{j+1}^{a*} \delta^{j+1}, \\ \hat{D}_{j+1}^b &\leq D_{j+1}^b \delta = \max\{D_j^b, C_j^b + p_{j+1} + q_{j+1}\} \delta \leq \max\{D_j^{b*} \delta^j, C_j^{b*} \delta^j + p_{j+1} + q_{j+1}\} \delta \\ &\leq \max\{D_j^{b*}, C_j^{b*} + q_{j+1}\} \delta^{j+1} = D_{j+1}^{b*} \delta^{j+1}, \\ \hat{D}_{j+1}^a &\leq D_{j+1}^a \delta = D_j^a \delta \leq D_j^{a*} \delta^{j+1} = D_{j+1}^{a*} \delta^{j+1}, \\ \hat{E}_{j+1} &\leq E_j \delta \leq E_j^* \delta^{j+1} \leq E_{j+1}^* \delta^{j+1}.\end{aligned}$$

In **Case (c)**, job  $j + 1$  is from  $\mathcal{J}_N$  and it is appended to the right schedule, resulting in a partial schedule as described in Eqs. (7). The recurrence relations are:  $C_{j+1}^b = C_j^b$ ,  $\Delta_{j+1} = \Delta_j$ ,  $t_{j+1}^a = t_j^a + p_{j+1}$ ,  $D_{j+1}^b = D_j^b$ ,  $D_{j+1}^a = \max\{D_j^a, t_{j+1}^a + q_{j+1}\}$ , and  $E_{j+1} = E_j$ .

Assume this 7-dimensional vector falls in the box  $(j + 1)\text{-}B_{r_1, r_2, r_3, r_4, r_5, r_6}$ . The 7-dimensional vector  $(j + 1; \hat{C}_{j+1}^b, \hat{\Delta}_{j+1}, \hat{t}_{j+1}^a, \hat{D}_{j+1}^b, \hat{D}_{j+1}^a, \hat{E}_{j+1})$  saved in this box by Algorithm 2 must satisfy  $\hat{C}_{j+1}^b \leq C_{j+1}^b \delta$ ,  $\hat{\Delta}_{j+1} \leq \Delta_{j+1}$ ,  $\hat{t}_{j+1}^a \leq t_{j+1}^a \delta$ ,  $\hat{D}_{j+1}^b \leq D_{j+1}^b \delta$ ,  $\hat{D}_{j+1}^a \leq D_{j+1}^a \delta$ , and  $\hat{E}_{j+1} \leq E_{j+1} \delta$ .

If the optimal vector  $(j; C_j^{b*}, \Delta_j^*, t_j^{a*}, D_j^{b*}, D_j^{a*}, E_j^*)$  leads to the optimal vector  $(j + 1; C_{j+1}^{b*}, \Delta_{j+1}^*, t_{j+1}^{a*}, D_{j+1}^{b*}, D_{j+1}^{a*}, E_{j+1}^*)$  also by appending job  $j + 1$  to the right schedule, we have the following recurrence relations:  $C_{j+1}^{b*} = C_j^{b*}$ ,  $\Delta_{j+1}^* = \Delta_j^*$ ,  $t_{j+1}^{a*} = t_j^{a*} + p_{j+1}$ ,  $D_{j+1}^{b*} = D_j^{b*}$ ,  $D_{j+1}^{a*} = \max\{D_j^{a*}, t_{j+1}^{a*} + q_{j+1}\}$ , and  $E_{j+1}^* = E_j^*$ .

From Eqs.(7), we have:  $C_{j+1}^b = C_j^b$ ,  $\Delta_{j+1} = \Delta_j$ ,  $t_{j+1}^a = t_j^a + p_{j+1}$ ,  $D_{j+1}^b = D_j^b$ ,  $D_{j+1}^a = \max\{D_j^a, t_{j+1}^a + q_{j+1}\}$ , and  $E_{j+1} = E_j$ . With  $\delta > 1$ , we can now bound the terms as follows:

$$\begin{aligned}\hat{C}_{j+1}^b &\leq C_{j+1}^b \delta = C_j^b \delta \leq C_j^{b*} \delta^{j+1} = C_{j+1}^{b*} \delta^{j+1}, \\ \hat{\Delta}_{j+1} &\leq \Delta_{j+1} = \Delta_j \leq \Delta_j^* = \Delta_{j+1}^*, \\ \hat{t}_{j+1}^a &\leq t_{j+1}^a \delta = (t_j^a + p_{j+1})\delta \leq (t_j^{a*} + p_{j+1})\delta^{j+1} = t_{j+1}^{a*} \delta^{j+1}, \\ \hat{D}_{j+1}^b &\leq D_{j+1}^b \delta = D_j^b \delta \leq D_j^{b*} \delta^{j+1} \leq D_{j+1}^{b*} \delta^{j+1}, \\ \hat{D}_{j+1}^a &\leq D_{j+1}^a \delta = \max\{D_j^a, t_j^a + p_{j+1} + q_{j+1}\} \delta \\ &\leq \max\{D_j^{a*} \delta^j, t_j^{a*} \delta^j + p_{j+1} + q_{j+1}\} \delta = D_{j+1}^{a*} \delta^{j+1}, \\ \hat{E}_{j+1} &\leq E_j \delta \leq E_j^* \delta^{j+1} \leq E_{j+1}^* \delta^{j+1}.\end{aligned}$$

In **Case (d)**, job  $j + 1$  is from  $\mathcal{J}_N$  and rejected, as described in Eqs.(8). The recurrence relations are:  $C_{j+1}^b = C_j^b$ ,  $\Delta_{j+1} = \Delta_j$ ,  $t_{j+1}^a = t_j^a$ ,  $D_{j+1}^b = D_j^b$ ,  $D_{j+1}^a = D_j^a$ , and  $E_{j+1} = E_j + e_{j+1}$ .

Assume this 7-dimensional vector falls in the box  $(j+1)\text{-}B_{r_1, r_2, r_3, r_4, r_5, r_6}$ . The 7-dimensional vector  $(j+1; \hat{C}_{j+1}^b, \hat{\Delta}_{j+1}, \hat{t}_{j+1}^a, \hat{D}_{j+1}^b, \hat{D}_{j+1}^a, \hat{E}_{j+1})$  saved in this box by Algorithm 2 must satisfy:  $\hat{C}_{j+1}^b \leq C_{j+1}^b \delta$ ,  $\hat{\Delta}_{j+1} \leq \Delta_{j+1}$ ,  $\hat{t}_{j+1}^a \leq t_{j+1}^a \delta$ ,  $\hat{D}_{j+1}^b \leq D_{j+1}^b \delta$ ,  $\hat{D}_{j+1}^a \leq D_{j+1}^a \delta$ , and  $\hat{E}_{j+1} \leq E_{j+1} \delta$ .

If the optimal vector  $(j; C_j^{b*}, \Delta_j^*, t_j^{b*}, D_j^{b*}, D_j^{a*}, E_j^*)$  leads to the optimal vector  $(j+1; C_{j+1}^{b*}, \Delta_{j+1}^*, t_{j+1}^{b*}, D_{j+1}^{b*}, D_{j+1}^{a*}, E_{j+1}^*)$  also by rejecting job  $j+1$ , then:  $C_{j+1}^{b*} = C_j^{b*}$ ,  $\Delta_{j+1}^* = \Delta_j^*$ ,  $t_{j+1}^{a*} = t_j^{a*}$ ,  $D_{j+1}^{b*} = D_j^{b*}$ ,  $D_{j+1}^{a*} = D_j^{a*}$ , and  $E_{j+1}^* = E_j^* + e_{j+1}$ .

From Eqs.(8), we have:  $C_{j+1}^b = C_j^b$ ,  $\Delta_{j+1} = \Delta_j$ ,  $t_{j+1}^a = t_j^a$ ,  $D_{j+1}^b = D_j^b$ ,  $D_{j+1}^a = D_j^a$ , and  $E_{j+1} = E_j + e_{j+1}$ . With  $\delta > 1$ , we can now bound the terms as follows:

$$\begin{aligned}\hat{C}_{j+1}^b &\leq C_{j+1}^b \delta = C_j^b \delta \leq C_j^{b*} \delta^{j+1} = C_{j+1}^{b*} \delta^{j+1}, \\ \hat{\Delta}_{j+1} &\leq \Delta_{j+1} = \Delta_j \leq \Delta_j^* = \Delta_{j+1}^*, \\ \hat{t}_{j+1}^a &\leq t_{j+1}^a \delta = (t_j^a + p_{j+1}) \delta \leq (t_j^{a*} + p_{j+1}) \delta^{j+1} = t_{j+1}^{a*} \delta^{j+1}, \\ \hat{D}_{j+1}^b &\leq D_{j+1}^b \delta = D_j^b \delta \leq D_j^{b*} \delta^{j+1} \leq D_{j+1}^{b*} \delta^{j+1}, \\ \hat{D}_{j+1}^a &\leq D_{j+1}^a \delta = D_j^a \delta \leq D_j^{a*} \delta^{j+1} = D_{j+1}^{a*} \delta^{j+1}, \\ \hat{E}_{j+1} &\leq E_{j+1} \delta = (E_j + e_{j+1}) \delta \leq (E_j^* \delta^j + e_{j+1}) \delta \leq E_{j+1}^* \delta^{j+1}.\end{aligned}$$

We therefore complete the proof of the induction statement.

Finally, we apply the inequality  $(1 + \varepsilon/2n)^n \leq 1 + \varepsilon$  for  $0 < \varepsilon < 1$  to bound the ratio  $\delta^n$ . That is, the sum of maximum deliver completion time of the original jobs and the accepted jobs from the new jobs, the weighted maximum completion time deviation and the total rejection cost of the output solution by Algorithm 2 is

$$\begin{aligned}&\max\{D_n^{b\varepsilon}, D_n^{a\varepsilon} + C_n^{b\varepsilon}\} + \mu \Delta_n^\varepsilon + E_n^\varepsilon \\ &\leq \max\{\hat{D}_n^b, \hat{D}_n^a + \hat{C}_n^b\} + \mu \hat{\Delta}_n + \hat{E}_n \\ &\leq \max\{D_n^{b*} \delta^n, (D_n^{a*} + C_n^{b*}) \delta^n\} + \mu \Delta_n^* + E_n^* \delta^n \\ &\leq (\max\{D_n^{b*}, (D_n^{a*} + C_n^{b*})\} + \mu \Delta_n^* + E_n^*) \delta^n \\ &\leq (1 + \varepsilon)(\max\{D_n^{b*}, (D_n^{a*} + C_n^{b*})\} + \mu \Delta_n^* + E_n^*).\end{aligned}$$

As for the time complexity, for each  $j$ , there are  $O(v_1 v_2 v_3 v_4 v_5 v_6)$  boxes associated with it. Each saved 7-dimensional vector  $(j; C_j^b, \Delta_j, l_j^b, D_j^b, D_j^a, E_j)$  takes  $O(1)$  time to assign job  $j+1$ , leading to at most four new 7-dimensional vector  $(j+1; C_{j+1}^b, \Delta_{j+1}, l_{j+1}^b, D_{j+1}^b, D_{j+1}^a, E_{j+1})$ , each updating the corresponding box associated with  $j+1$ . Thus, the running time is  $O(n v_1 v_2 v_3 v_4 v_5 v_6)$ . From the definitions of  $v_i$ 's, and noting that  $\log \delta = \log(1 + \varepsilon/2n) \geq \varepsilon/4n$ . We conclude that  $O(n v_1 v_2 v_3 v_4 v_5) \subseteq O(n^6 (\log M)^5 / \varepsilon^5)$ , where  $M = \max\{p_{\max}, e_{\max}, q_{\max}\}$ , which is in polynomial time.



## 6 Concluding Remarks

In this paper, we address the rescheduling problem on a single machine, focusing on the impact of newly arriving jobs with the option for job rejection. The primary scheduling objective is to minimize the maximum delivery completion time. We propose a novel general rescheduling model where the total cost includes the scheduling objective for the original jobs and the newly accepted jobs, a disruption penalty, and the total rejection cost, subject to a maximum allowable disruption threshold. To solve this problem, we derive a pseudo-polynomial-time dynamic programming exact algorithm. Additionally, we present a fully polynomial-time approximation scheme (FPTAS). Future research could explore extensions of this model to different machine environments, such as flow shops or parallel machines, or consider other types of disruptions that may arise in more complex scheduling scenarios.

**Author Contributions** The authors make their contributions equally.

**Conflicts of Interest** The authors declare no conflict of interest.

## References

- [1] Bean, J.C., Birge, J.R., Mittenthal, J., Noon, C.E.: Matchup scheduling with multiple resources, release dates and disruptions. *Operations Research*. **39**(3), 470–483 (1991)
- [2] Clausen, J., Hansen, J., Larsen, J., Larsen, A.: Disruption management. *OR/MS Today* 40–43 (2001)
- [3] Dahal, K., Al-Arfaj, K., Paudyal, K.: Modelling generator maintenance scheduling costs in deregulated power markets. *European Journal of Operational Research*. **240**(2), 551–561 (2015)
- [4] Garey, M.R., Johnson, D.S.: *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, New York (1979)
- [5] Guo, Y., Huang, M., Wang, Q., Leon, V.J.: Single-machine rework rescheduling to minimize total waiting time with fixed sequence of jobs and release times. *IEEE Access*. **9**, 1205–1218 (2019)
- [6] Graham, R.L., Lawler, E.L., Lenstra, J.K., Rinnooy Kan, A.H.G.: Optimization and approximation in deterministic sequencing and scheduling: a survey. *Annals of Discrete Mathematics*. **5**(1), 287–326 (1979)
- [7] Hall, N.G., Potts, C.N.: Rescheduling for job unavailability. *Operations Research*. **58**, 746–755 (2010)
- [8] Hall, N.G., Potts, C.N.: Rescheduling for new orders. *Operations Research*. **52**, 440–453 (2004)
- [9] Hall, N.G., Liu, Z., Potts, C.N.: Rescheduling for multiple new orders. *INFORMS Journal on Computing*. **19**(4), 633–645 (2007)
- [10] Liu, Z., Ro, Y.K.: Rescheduling for machine disruption to minimize makespan and maximum lateness. *Journal of Scheduling*. **17**(4), 339–352 (2014)
- [11] Lendl, S., Pferschy, U., Renner, E.: Rescheduling with New Orders Under Bounded Disruption. *INFORMS Journal on Computing*. **36**(6), 1654–1675 (2024)
- [12] Fang, K., Luo, W., Pinedo, M.L., Jin, M., Lu, L.: Rescheduling for new orders on a single machine with rejection. *Journal of the Operational Research Society*. **75**(2), 346–360 (2024)
- [13] Ouelhadj, D., Petrovic, S.: A survey of dynamic scheduling in manufacturing systems. *Journal of Scheduling*. **12**, 417–431 (2009)
- [14] Teghem, J., Tuytens, D.: A bi-objective approach to reschedule new jobs in a one machine model. *International Transactions in Operational Research*. **21**(6), 871–898 (2014)
- [15] Renner, E., Salassa, F., T'kindt, V.: Single machine rescheduling for new orders with maximum lateness minimization. *Computers & Operations Research*. **144**, 105815 (2022)
- [16] Renner, E., Salassa, F., T'kindt, V.: Single machine rescheduling for new orders: Properties and complexity results (2023). [arxiv:2307.14876](https://arxiv.org/abs/2307.14876)
- [17] Vieira, G.E., Herrmann, J.W., Lin, E.: Rescheduling manufacturing systems: a framework of strategies, policies, and methods. *Journal of Scheduling*. **6**(1), 39–62 (2003)

- [18] Wang, D., Liu, F., Jin, Y.: A multi-objective evolutionary algorithm guided by directed search for dynamic scheduling. *Computers & Operations Research*. **9**, 279–290 (2016)
- [19] Yu, G., Ello, M., Song, G., Mccowan, S.M., White, A.: A new era for crew recovery at continental airlines. *Interfaces*. **33**(1), 5–22 (2003)
- [20] Zweben, M., Davis, E., Daun, B., Deale, M.J.: Scheduling and rescheduling with iterative repair. *IEEE Transactions on Systems Man & Cybernetics*. **23**(6), 1588–1596 (1992)
- [21] Zhao, Q., Lu, L., Yuan, J.: Rescheduling with new orders and general maximum allowable time disruptions. *4OR*. **14**, 261–280 (2016)
- [22] Zhang, X., Lin, W.C., Wu, C.C.: Rescheduling problems with allowing for the unexpected new jobs arrival. *Journal of Combinatorial Optimization*. **43**(3), 630–645 (2022)